

STUBBORN

Don't compromise on your look

Documentation technique de l'application

Site e-commerce développé avec Symfony 6.4

Projet réalisé dans le cadre du devoir CEF
« Réaliser un site e-commerce avec Symfony »

Sommaire

1. Présentation du projet
2. Stack technique et architecture
3. Modèle de données
4. Authentification et sécurité
5. Fonctionnalités de l'application
6. Intégration du paiement Stripe
7. Tests unitaires
8. Installation et exécution du projet en local
9. Structure du dépôt GitHub

1. Présentation du projet

Stubborn est une marque de sweat-shirts fictive. Ce projet consiste à développer, pour le compte de cette marque, une boutique en ligne permettant à un client de consulter le catalogue, de créer un compte, d'ajouter des articles à son panier et de régler sa commande via une solution de paiement en ligne (Stripe). Un espace d'administration permet par ailleurs à un utilisateur disposant du rôle administrateur de gérer le catalogue de produits (ajout, modification, suppression).

Informations sur la société cliente

Champ	Valeur
Nom de la société	Stubborn
Adresse	Piccadilly Circus, London W1J 0DA, Royaume-Uni
Adresse de contact	stubborn@blabla.com
Slogan	Don't compromise on your look

2. Stack technique et architecture

Composant	Choix technique
Framework backend	Symfony 6.4 (LTS)
Langage	PHP 8.2
Base de données	MySQL 9.1
ORM	Doctrine ORM
Moteur de templates	Twig
Authentification	Symfony Security Bundle
Vérification d'email	SymfonyCasts VerifyEmailBundle
Paieement	Stripe Checkout (mode test / sandbox)
Gestion des versions	Git / GitHub
Tests	PHPUnit 11
Jeu de données de démonstration	Doctrine Fixtures Bundle

Architecture générale

L'application suit l'architecture MVC standard de Symfony. Les contrôleurs (src/Controller) reçoivent les requêtes HTTP et s'appuient sur des repositories Doctrine pour l'accès aux données, ainsi que sur des services métier dédiés (CartService, StripeService) pour la logique applicative. Les vues sont rendues avec Twig et héritent toutes d'un layout commun (templates/base.html.twig) qui gère le header, le menu dynamique et le footer.

Dossier	Rôle
src/Controller	Contrôleurs (une classe par grande fonctionnalité : Home, Product, Cart, Checkout, Admin, Security, Registration)
src/Entity	Entités Doctrine : User, Sweatshirt, Stock
src/Repository	Requêtes personnalisées (ex. filtre par fourchette de prix)
src/Service	Services métier : CartService (panier en session), StripeService (paiement)
src/Form	Formulaire d'inscription (RegistrationFormType)
src/DataFixtures	Jeu de données de démonstration (10 sweat-shirts + compte admin)
templates/	Vues Twig, organisées par domaine (home, product, cart, checkout, admin, registration, security)
assets/styles/app.css	Feuille de style unique de l'application
tests/Service	Tests unitaires du panier et de la logique de paiement

3. Modèle de données

Entité User

Représente un utilisateur, qu'il soit client ou administrateur. Le rôle par défaut attribué à l'inscription est `ROLE_CLIENT` ; le rôle `ROLE_ADMIN` hérite des droits de `ROLE_CLIENT` (hiérarchie de rôles Symfony).

Champ	Type	Description
id	int	Identifiant unique
email	string	Identifiant de connexion, unique
password	string	Mot de passe haché
name	string	Nom de l'utilisateur
deliveryAddress	string	Adresse de livraison
roles	json	Rôles Symfony (<code>ROLE_CLIENT</code> / <code>ROLE_ADMIN</code>)
isVerified	bool	Vrai une fois l'email confirmé

Entité Sweatshirt

Champ	Type	Description
id	int	Identifiant unique
name	string	Nom du sweat-shirt
price	float	Prix, identique quelle que soit la taille
image	string	Nom du fichier image (public/uploads/sweatshirts/)
featured	bool	Mis en avant sur la page d'accueil
stocks	Collection<Stock>	Relation OneToMany vers Stock

Entité Stock

Chaque sweat-shirt possède exactement une ligne de stock par taille disponible (XS, S, M, L, XL), ce qui permet de gérer une quantité indépendante par taille tout en gardant un prix unique par produit.

Champ	Type	Description
id	int	Identifiant unique
size	string	Taille (XS, S, M, L, XL)
quantity	int	Quantité disponible pour cette taille
sweatshirt	Sweatshirt	Relation ManyToOne vers Sweatshirt

Schéma relationnel simplifié

Une entité `Sweatshirt` possède plusieurs entités `Stock` (une par taille) : relation One-to-Many / Many-to-One.

Sweatshirt (1)	Stock (N)
id, name, price, image, featured	id, size, quantity, sweatshirt_id (clé étrangère)

L'entité `User` est indépendante des deux autres (aucune relation directe avec `Sweatshirt` ou `Stock`).

4. Authentification et sécurité

- Authentification gérée par le composant Security de Symfony, avec un provider Doctrine basé sur l'entité User (identifiant : email).
- Deux rôles : ROLE_CLIENT (par défaut à l'inscription) et ROLE_ADMIN (hérite de ROLE_CLIENT via role_hierarchy).
- Page de connexion : route /login (formulaire Symfony form_login).
- Page d'inscription : route /register. Le formulaire collecte nom, email, adresse de livraison et mot de passe (avec confirmation).
- À l'inscription, un email de confirmation est envoyé (via SymfonyCasts VerifyEmailBundle) contenant un lien signé menant à la route /verify/email. Une fois le lien cliqué, le champ isVerified passe à vrai et l'utilisateur est redirigé vers la page d'accueil.
- L'utilisateur est automatiquement connecté immédiatement après son inscription.
- Le back-office (route /admin et sous-routes) est protégé par un contrôle d'accès (access_control dans security.yaml et attribut #[IsGranted('ROLE_ADMIN')] sur le contrôleur), garantissant qu'un client authentifié ne peut pas y accéder.

Menu dynamique

Le header (templates/base.html.twig) affiche un menu différent selon l'état de connexion, conformément au cahier des charges :

Utilisateur non connecté	Utilisateur connecté
Accueil / S'inscrire / Se connecter	Accueil / Boutique / Panier / Se déconnecter

Le lien correspondant à la page courante est affiché en gras et souligné (classe CSS .active).

5. Fonctionnalités de l'application

Route	Méthode	Description
/	GET	Page d'accueil : présentation de la société et 3 sweat-shirts mis en avant
/login	GET / POST	Formulaire de connexion
/register	GET / POST	Formulaire d'inscription
/verify/email	GET	Validation du lien de confirmation d'email
/products	GET	Liste de tous les sweat-shirts, filtrable par fourchette de prix
/product/{id}	GET	Fiche détaillée d'un sweat-shirt, choix de la taille
/cart	GET	Contenu du panier et total
/cart/add/{id}	POST	Ajout d'un article (avec taille) au panier
/cart/remove/{id}/{size}	POST	Retrait d'un article du panier
/checkout	GET	Création de la session de paiement Stripe et redirection
/checkout/success	GET	Confirmation de commande, vidage du panier
/admin	GET	Back-office (liste des sweat-shirts, réservé ROLE_ADMIN)
/admin/add	POST	Ajout d'un sweat-shirt
/admin/update/{id}	POST	Modification d'un sweat-shirt (prix, image, mise en avant, stock)
/admin/delete/{id}	POST	Suppression d'un sweat-shirt

Le panier

Le panier est géré en session via le service CartService (src/Service/CartService.php), sous la forme d'un tableau associatif clé = "idSweatshirt_taille" et valeur = quantité. Cette approche permet de distinguer deux tailles différentes d'un même produit comme deux lignes distinctes du panier, et de recalculer automatiquement le total à chaque ajout ou retrait.

6. Intégration du paiement Stripe

- Un compte Stripe a été créé et l'environnement de test (sandbox) est utilisé exclusivement pour ce projet.
- Le SDK officiel stripe/stripe-php a été installé via Composer.
- Un service dédié, StripeService (src/Service/StripeService.php), encapsule la logique d'appel à l'API Stripe : il construit la liste des articles (line items, montants convertis en centimes) puis crée une session Stripe Checkout.
- Le contrôleur CheckoutController redirige l'utilisateur vers la page de paiement hébergée par Stripe (route /checkout), puis le récupère sur la route /checkout/success après un paiement accepté, où le panier est vidé.
- Les clés API (clé publique et clé secrète de test) sont stockées dans le fichier .env, non versionné, et injectées dans le service via la configuration Symfony (config/services.yaml).

Réaliser un achat test

Pour simuler un paiement réussi en mode développement, Stripe fournit une carte bancaire de test :

Champ	Valeur de test
Numéro de carte	4242 4242 4242 4242
Date d'expiration	N'importe quelle date future
CVC	N'importe quel code à 3 chiffres

Aucune transaction réelle n'est effectuée : ces paiements n'apparaissent que dans le tableau de bord Stripe en mode test.

7. Tests unitaires

Les tests sont écrits avec PHPUnit et se trouvent dans le dossier tests/Service. Ils couvrent les deux fonctionnalités demandées par le cahier des charges : le panier et le règlement d'une commande.

CartServiceTest (7 tests)

- Le panier est vide par défaut.
- L'ajout d'un article fonctionne.
- Ajouter deux fois le même article/taille incrémente la quantité.
- Deux tailles différentes du même produit créent deux lignes distinctes.
- Le retrait d'un article fonctionne.
- Le total est calculé correctement (quantités × prix, tous articles confondus).
- Vider le panier le rend bien vide.

StripeServiceTest (3 tests)

- La construction des articles Stripe (line items) fonctionne pour un article unique, avec conversion correcte du prix en centimes.
- La construction fonctionne pour plusieurs articles de tailles et quantités différentes.
- Un panier vide produit une liste d'articles vide, sans erreur.

Exécution des tests

```
php bin/phpunit
```

Résultat obtenu lors de la rédaction de cette documentation :

```
OK (10 tests, 20 assertions)
```

8. Installation et exécution du projet en local

Prérequis

- PHP 8.2 ou supérieur, avec les extensions : zip, intl, openssl, mbstring, pdo_mysql, fileinfo, curl
- Composer
- Serveur MySQL (8.0 ou supérieur)
- Git

Étapes d'installation

```
# 1. Cloner le dépôt
git clone https://github.com/AdelC57/stubborn.git
cd stubborn

# 2. Installer les dépendances PHP
composer install

# 3. Configurer la base de données dans le fichier .env
DATABASE_URL="mysql://root:@127.0.0.1:3306/stubborn?serverVersion=8.0"

# 4. Configurer les clés Stripe (mode test) dans le fichier .env
STRIPE_PUBLIC_KEY=pk_test...
STRIPE_SECRET_KEY=sk_test...

# 5. Créer la base de données et exécuter les migrations
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate

# 6. Charger le jeu de données de démonstration
php bin/console doctrine:fixtures:load

# 7. Placer les images des produits
# Copier les fichiers dans public/uploads/sweatshirts/

# 8. Lancer le serveur
php -S 127.0.0.1:8000 -t public public/router.php

# 9. Ouvrir l'application
http://127.0.0.1:8000/
```

Comptes de démonstration

Rôle	Email	Mot de passe
Administrateur	admin@stubborn.com	admin1234
Client	à créer via /register	—

Exécuter les tests unitaires

```
php bin/phpunit
```

9. Structure du dépôt GitHub

Le projet est versionné avec Git et hébergé sur GitHub. Le fichier `.gitignore` exclut notamment le dossier `vendor/`, le fichier `.env` (contenant les clés Stripe et les identifiants de base de données) et le cache Symfony (`var/`), conformément aux bonnes pratiques.

Élément	Détail
Dépôt	https://github.com/AdelC57/stubborn
Branche principale	main
Fichiers exclus (<code>.gitignore</code>)	<code>.env</code> , <code>/vendor/</code> , <code>/var/</code> , <code>/node_modules/</code>
Documentation	Le présent document, exporté en PDF